SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1714**

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I S.PRAVEEN (192524239) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Reverse Engineering a Warehouse Robotics Coordination System

Working backward from the observed behaviour of a fulfillment-center robot fleet — hundreds of robots picking, packing, and moving inventory without collisions — to infer the underlying architecture: how PySpark processes sensor and task-queue logs at scale, how RDD operations give the system fault tolerance, and how intelligent agents coordinate task allocation and real-time pathfinding.

1. Reverse-Engineering Approach

To reverse engineer the system, we start from three observable facts: (a) hundreds of robots emit continuous sensor telemetry (position, LIDAR, battery) and task events, (b) the fleet never stalls even when individual robots or servers fail, and (c) robots are re-tasked and re-routed within milliseconds of a warehouse-floor change. These behaviours point to a **distributed, streaming, fault-tolerant batch/stream hybrid** built on PySpark for the data layer, coordinated by a set of specialized agents for real-time decision-making — the architecture inferred below.

2. Inferred System Architecture

Sensor and task-queue logs from every robot are inferred to flow through a streaming ingestion layer into a PySpark cluster for RDD-based processing, whose output state feeds a multi-agent coordination layer that issues real-time task and motion commands back to the fleet:

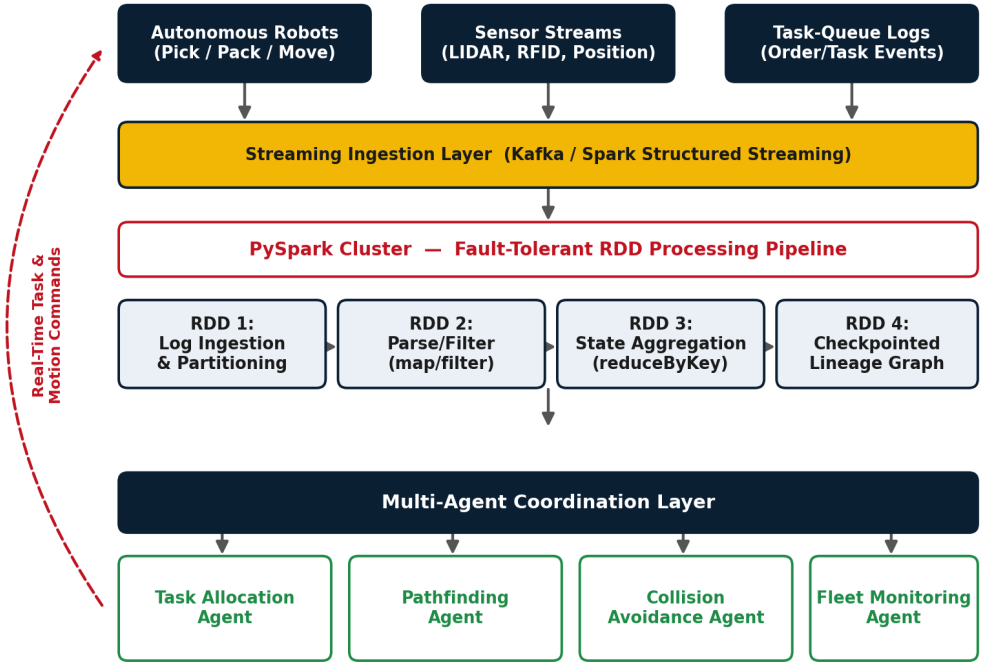

Figure 1: Reverse-engineered architecture of the warehouse robotics system

3. How PySpark Processes Sensor & Task-Queue Logs

The volume of telemetry (hundreds of robots reporting position/status several times a second, plus a constant stream of pick/pack/move task events) rules out a single-machine log processor. The evidence — near-instant fleet-wide reactions and linear scaling as more robots are added — indicates a Spark Structured Streaming job feeding a chain of RDD transformations:

- **Log Ingestion RDD:** Raw JSON/binary telemetry and task events are read from a message queue (e.g. Kafka) and converted into an RDD, automatically partitioned by robot ID or warehouse zone so load is spread evenly across the cluster.
- **map()/filter() — Parsing & Cleaning:** Each partition parses raw sensor payloads into structured records (position, heading, battery, task status) and filters out malformed or stale packets in parallel.
- **reduceByKey() — State Aggregation:** Per-robot and per-zone state (occupied cells, active tasks, queue depth) is aggregated by key so the coordination layer always reads a consolidated, de-duplicated view rather than raw per-packet noise.
- **groupByKey()/join():** Task-queue events are joined with robot-status RDDs to match pending pick/pack jobs against currently idle or nearby robots, forming the candidate list the Task Allocation Agent consumes.
- **checkpoint():** Long RDD lineage chains from continuous streaming are periodically checkpointed to stable storage, bounding recovery time and preventing an ever-growing dependency graph.

4. How RDD Operations Support Fault Tolerance

The fleet's ability to keep operating through individual node or robot failures is explained by RDDs' core fault-tolerance mechanism: each RDD records its **lineage** — the sequence of transformations used to derive it — rather than immediately replicating data. If a Spark executor processing a shard of robot telemetry crashes mid-job, only the lost partitions are recomputed from lineage and re-run on a healthy node; the rest of the fleet's state is untouched and no full restart is required. Combined with periodic checkpointing (point 3) to cap recomputation depth, and the immutability of RDDs (so partial/corrupt writes never poison shared state), this gives the warehouse the observed property that a server or robot failure causes a brief local delay, never a fleet-wide stoppage.

5. Multi-Agent Task Allocation & Real-Time Pathfinding

The instantaneous, collision-free re-tasking observed on the floor is best explained by a layer of specialized agents consuming the aggregated RDD state:

- **Task Allocation Agent:** Matches pending tasks from the task-queue RDD to the nearest suitable idle robot (based on battery, load capacity, and distance), issuing new pick/pack/move assignments as warehouse orders arrive.
- **Pathfinding Agent:** Computes an optimal route for each assigned robot across the warehouse grid (e.g. A*/Dijkstra-based) using the current occupancy map derived from aggregated position RDDs, recalculating instantly whenever the map changes.
- **Collision Avoidance Agent:** Continuously cross-checks planned paths of nearby robots in real time and issues local speed/route adjustments the instant two paths are predicted to intersect, explaining why robots never collide even in dense aisles.

- **Fleet Monitoring Agent:** Watches robot health (battery, faults, task backlog) and reroutes tasks away from a robot that is about to fail or run low on charge, before it disrupts the fleet.

These agents operate on the shared, continuously-updated RDD-derived state rather than raw logs, which is why task allocation and path recalculation happen in real time — the expensive log processing has already been done upstream by the PySpark pipeline, leaving the agents to make fast decisions on a small, current state snapshot.

6. Conclusion

Reverse engineering the fleet's observed behaviour — scale, resilience to failure, and real-time collision-free coordination — converges on an architecture where PySpark's RDD-based pipeline handles the heavy, fault-tolerant processing of sensor and task-queue logs, while a lean layer of task-allocation, pathfinding, collision-avoidance, and fleet-monitoring agents consumes that processed state to make split-second coordination decisions across hundreds of robots.